

SC4063: Network Security

Week 3 : Zero Trust Network Architecture

Chua Zong Fu





Content

- Modern Network Security
- Discussion



Modern Network Security



NIST Cybersecurity Framework (CSF) 2.0

The **NIST Cybersecurity Framework (CSF) 2.0** is a **risk-based, outcome-driven framework** that helps organizations **govern, manage, and reduce cybersecurity risk** across technology, people, and processes.

Core Functions (CSF 2.0)

1. **Govern** – Establish oversight, roles, and risk strategy
2. **Identify** – Understand assets, risks, and dependencies
3. **Protect** – Implement safeguards to limit impact
4. **Detect** – Identify cybersecurity events promptly
5. **Respond** – Contain and mitigate incidents
6. **Recover** – Restore capabilities and resilience

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Now let's anchor everything we've been discussing to a framework that actually governs how security is done in the real world, and that's the NIST Cybersecurity Framework, specifically version 2.0. This is not a checklist, and it is not a compliance standard. It's a way of thinking about cybersecurity as a risk management problem, not just a technical one.

CSF 2.0 introduces an important evolution by explicitly adding the **Govern** function. This recognizes a reality that many breaches have already taught us: you can have excellent technical controls and still fail if accountability, ownership, and risk decisions are unclear. Governance is about who owns risk, how decisions are made, and how security aligns with business objectives. Without this layer, security becomes reactive and fragmented.

The remaining five functions form a lifecycle. You identify what you have and what matters. You protect what you can through architecture and controls. You detect what inevitably slips through. You respond to contain damage. And you recover to restore operations and improve resilience. These are not sequential steps, they operate continuously and in parallel.

From a network security perspective, this framework is extremely powerful. Firewalls, segmentation, Zero Trust, and identity controls all sit primarily in Protect. Telemetry, flow logs, DNS monitoring, and NDR belong in Detect.

Incident response workflows and automation map to Respond. Backup strategies, immutable infrastructure, and recovery planning live in Recover. And threat modeling, asset discovery, and exposure management are core to Identify. What CSF gives us is a **shared language**. Engineers can map controls, architects can design systems, executives can understand risk posture, and regulators can assess maturity, all without arguing over tools. Throughout this course, every major topic you encounter can be placed somewhere in this framework. If you cannot explain where a control fits in CSF, that's a signal that the control may be poorly understood or poorly justified.

The key takeaway is this: modern network security is not about deploying more technology. It's about making deliberate, risk-informed decisions across the entire security lifecycle. CSF 2.0 is the structure that allows those decisions to be made coherently and defended credibly.



Network Security Is a System, Not a Tool

- **Legacy View:** Security = Appliances (Firewalls, IDS, WAF).
- **Modern View:** Security = A quality of the architecture (Resilience, Observability).
- Stallings Link: Distinction between Security Mechanisms (tools) and Security Services (outcomes).
- Case Study: [Netflix Chaos Monkey](#)
 - Concept: Chaos Engineering.
 - Method: Randomly terminating instances in production to ensure the system (network + app) heals itself. [T1485]
 - Security Value: Proves availability (CIA Triad) without relying on a single "box" to stay up.

- Modern security is code injected into the infrastructure, not a box rack-mounted next to it.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In this session, we are going to define what 'Modern Network Security' actually is. If you ask a junior engineer, they might point to a Palo Alto firewall or a CrowdStrike agent. But as veterans, we know that tools are not security. Tools are mechanisms. Security is a state of being.

Stallings makes a critical distinction in Chapter 1 between 'Security Mechanisms', the specific methods like encryption or digital signatures, and 'Security Services', the outcomes like Confidentiality and Integrity. In the modern era, we must view network security as a system, not a collection of boxes. If your firewall goes down, does your security posture collapse? If the answer is yes, you have built a fragile system.

Consider Netflix. They pioneered 'Chaos Engineering' with Chaos Monkey. They deliberately break their own network in production. They terminate servers and sever connections to ensure the system can route around failure. This is the ultimate test of Availability, the 'A' in the CIA triad. Modern security is about designing architectures that assume the tool will fail, the network will be hostile, and the adversary is already inside. We are moving from 'buying security' to 'architecting resilience'.



Control Plane vs. Data Plane

- **Data Plane:** The packets moving user traffic (e.g. HTTP requests, Database queries).
- **Control Plane:** The logic deciding where packets move (e.g. BGP, DNS, IAM, Kubernetes API).
- **The Threat:** Attackers target the Control Plane to redirect or fundamentally alter the Data Plane.
- **Case Study:** [Facebook BGP Outage](#) (2021)
 - Incident: A configuration error in the BGP backbone (Control Plane).
 - Result: Facebook withdrew its DNS routes. The entire platform (Data Plane) disappeared from the internet [T1498]
 - Lesson: If you lose the Control Plane, the Data Plane is irrelevant.

- Adversaries don't just want to read your traffic; they want to route it.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

To understand modern security, you must distinguish between the Data Plane and the Control Plane. The Data Plane is the highway, it's where the cars (packets) drive. The Control Plane is the GPS system and the traffic lights, it decides where the cars go.

In traditional security, we spent 90% of our time watching the Data Plane, inspecting packets for malware. But the most devastating attacks happen in the Control Plane. If I can compromise your BGP routes, your DNS records, or your Identity Provider, I don't need to hack your server; I simply tell the internet that your server is now my server.

The Facebook outage of 2021 was a prime example of a Control Plane failure. It wasn't a hack, but it demonstrated the physics of the problem. A bad BGP configuration update wiped Facebook's routes from the internet. Their servers were fine, their code was fine, but they ceased to exist because the Control Plane failed. In a cloud environment, IAM (Identity and Access Management) is the Control Plane. If an attacker compromises an IAM role, they control the routing of data. We must shift our monitoring focus up to the Control Plane.



Module 1 : Identity



Zero Trust Architecture

- The Definition ([NIST 800-207](#)):
 - Core Tenet: "**Never Trust, Always Verify.**"
 - Shift: Moving the **Policy Decision Point** (PDP) closer to the **Policy Enforcement Point** (PEP)
 - Scope: Not just users; applies to workloads, devices, and data flows.
- The Three Pillars of Verification:
 - **Identity**: Is this user/service who they say they are?
 - **Context**: Is the device healthy? Is the location valid?
 - **Privilege**: Does the user need this specific resource?
- Case Study: [Google BeyondCorp](#)
- Origin: Created post-Aurora attack to remove privileged intranets.
- Outcome: All employees work from "untrusted" networks (like Starbucks) with the same security as the office.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

We've torn down the old models; now we are going to build new ones. It starts with Zero Trust. This is the most abused buzzword in our industry, so let's be precise. We rely on NIST 800-207.

Zero Trust is often misunderstood as a product or a perimeter replacement, and it is neither. According to NIST 800-207, Zero Trust is an architectural model that removes the assumption that anything is trusted simply because it is on the internal network. Historically, once you were "inside," you were trusted. Zero Trust explicitly rejects that assumption.

The core idea is continuous verification. Every access request is evaluated based on who is making the request, what device they are using, the context of that request, and whether they actually need the resource. This is why the architecture emphasizes policy decision points and policy enforcement points. The closer enforcement is to the resource, the smaller the blast radius when something goes wrong.

It is also important to understand that Zero Trust is not just about users. Modern environments are dominated by machine-to-machine communication, APIs, and workloads. These entities require the same scrutiny as human users. If you only apply Zero Trust to people and ignore services and data flows, you have not implemented Zero Trust.

Google's BeyondCorp is frequently cited because it demonstrates this model at scale. After the Aurora attacks, Google removed the concept of a privileged internal network. Employees could access internal systems from anywhere, including public networks, because access decisions were based on identity, device health, and policy enforcement rather than network location. The most important takeaway for modern network security is this: Zero Trust does not eliminate networks, firewalls, or segmentation. It **changes what we trust**. The network becomes a transport layer, not a security boundary. Trust decisions move upward to identity, policy, and context, which is exactly why Zero Trust is foundational to modern cloud and hybrid architectures.



ZTNA vs. VPN: The Architectural Split

- VPN (Network Centric):
 - Model: Connect user to the Network.
 - Risk: Lateral Movement. Once inside, the user has L3 visibility.
- ZTNA (Application Centric):
 - Model: Connect user to the Application.
 - Mechanism: Outbound-only connectors. No inbound open ports.
 - Benefit: The application is "Dark" to the internet.
- Case Study: [Zscaler / Pulse Secure CVEs](#)
 - Contrast: While VPN concentrators (Pulse Secure) faced critical RCE vulnerabilities, ZTNA architectures were unaffected because they expose no listening surface. [T1190]

This slide captures a shift that is often misunderstood as a technology upgrade, when in reality it is a change in how access is fundamentally modeled.

Traditional VPNs are network-centric. When a user authenticates, they are effectively placed inside the network. Even if access controls exist, the user can see network routes, services, and sometimes entire address ranges. From an attacker's perspective, compromising VPN access is extremely valuable because it restores the same trust assumptions that internal users enjoy.

ZTNA flips this model. Instead of connecting users to a network, it connects them to a specific application. There is no broad Layer 3 visibility, no routable access to subnets, and no implicit trust because of location. Applications initiate outbound connections to a trusted broker, which means there are no inbound services listening on the Internet. This single architectural decision dramatically reduces the exposed attack surface.

It's important to understand that ZTNA is not just a more secure VPN. It removes an entire class of network-level risks. Lateral movement becomes significantly harder because users never gain network adjacency to other systems. Discovery is constrained because services are not visible. Even if credentials are compromised, the attacker is limited to exactly what policy allows.

The takeaway here is intentionally blunt: VPNs solve connectivity; ZTNA solves

access. Modern network security prioritizes reducing implicit trust and minimizing blast radius. When you compare these two models, you are not choosing between products. You are choosing between extending the network perimeter outward or dissolving it altogether in favor of identity- and policy-driven access.



Identity as Routing Logic

- The Death of IP-Based Routing:
 - Old Logic: Allow 192.168.1.10 to 10.0.0.5.
 - New Logic: Allow Service=Frontend to Service=Backend if Cert is Valid.
- Mechanism: Service Mesh (Istio, Linkerd) and Sidecars.
- Case Study: [Google BeyondCorp](#)
 - Shift: Moved access decisions from the VPN concentrator to a Reverse Proxy.
 - Criteria: Access granted based on User Identity + Device Certificate, not network location.
 - Result: Zero Trust Network Access (ZTNA).

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Historically, networks made decisions based on IP addresses. If traffic came from the right subnet, it was trusted. That made sense when networks were static, devices were fixed, and everything lived in a data center. None of those assumptions hold today.

In modern environments, IP addresses are transient. Containers spin up and disappear. Cloud workloads move. Users work from anywhere. As a result, IP-based trust becomes brittle and dangerous. If an attacker gains a foothold anywhere inside the network, they inherit the same routing privileges as legitimate systems.

Identity-based routing changes the question entirely. Instead of asking, “Where is this packet coming from?” the system asks, “Who or what is making this request, and should it be allowed?” Identity is cryptographically asserted, policy is centrally defined, and enforcement happens as close to the resource as possible.

Technologies like service meshes and sidecars operationalize this idea. They sit alongside applications and enforce identity-based policy on every connection, often using mutual authentication. The network becomes a transport mechanism, not a trust boundary.

Google’s BeyondCorp demonstrates this shift at scale. By removing VPN-based

network trust and replacing it with identity and device verification at the application layer, Google eliminated the concept of a privileged internal network. Access is evaluated continuously, regardless of where the user or workload is located.

The key takeaway is simple but powerful. Modern network security is not about controlling routes between subnets. It is about controlling relationships between identities. Once you understand that, Zero Trust stops sounding like a buzzword and starts looking like a natural evolution of how networks must operate in cloud and hybrid environments.



Identity Governance and Administration

- The Lifecycle Problem: Access risk evolves as identities change:
 - Joiner – Initial access provisioning
Risk: Excessive default permissions
 - Mover – Role or responsibility changes
Risk: Privilege accumulation and access drift
 - Leaver – Access revocation
Highest risk: Dormant or forgotten credentials
- Entitlement Management:
 - Users and service accounts often accumulate more permissions than required. Over-provisioning expands blast radius. Privilege creep undermines Zero Trust assumptions
- Case Study: [Colonial Pipeline \(Leaver Failure\)](#)
 - Root Cause: The VPN account belonged to a former employee. The "Leaver" process failed to deprovision the account. [T1078.003]

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

We said Identity is the new perimeter. If that's true, then Identity Governance and Administration is your firewall management console.

The biggest risks are 'Movers' and 'Leavers.' When an employee moves from Engineering to Sales, do they lose their Engineering access? Usually not. This is 'Privilege Creep.' And 'Leavers', people who quit, are critical.

From a network security perspective, this matters because Zero Trust assumes identity decisions are correct. If identity governance fails, Zero Trust enforcement becomes meaningless. A perfectly enforced policy is still a failure if the wrong identity is granted access.

The Colonial Pipeline incident illustrates this clearly. There was no sophisticated exploit. There was no lateral movement required. A valid VPN account belonging to a former employee was enough. The failure was not in encryption, routing, or firewalls. It was in identity lifecycle governance.

This is why IGA is not an HR process or a compliance checkbox. It is a security control. It defines who exists in the system, what they are allowed to do, and whether that access still makes sense. In modern architectures, identity has replaced the network perimeter, and IGA is the mechanism that ensures that perimeter does not silently erode over time.

The takeaway is simple: Zero Trust without strong identity governance is an

illusion. You can verify continuously, enforce perfectly, and monitor everything, but if identities are wrong, the system will fail quietly and catastrophically.



IAM Policies: Least Privilege at Scale

- The Permission Boundary:
 - Legacy: "Admins" have *.* access.
 - Modern: **Attribute-Based Access Control** (ABAC).
- **Concept:** Permissions are granted based on **Tags, Time, and Source**.
- Case Study: [AWS IAM Access Analyzer](#)
- Function: **Mathematically** proving which resources are accessible from the internet.

In the cloud, IAM is the network. Your 'Firewall' rules are actually IAM Policies. The principle is Least Privilege. But doing this manually is hard. We use tools like IAM Access Analyzer. It uses automated reasoning (math) to look at your policies and tell you: 'This role allows access to S3 from the public internet.' It identifies unintended escalation paths that a human eye would miss in a 1,000-line JSON policy file.



Just-in-Time Access

- **Zero Standing Privileges:**
 - Problem: Admins having 24/7 "God Mode" access.
 - Solution: Just-in-Time (JIT) Access.
- **Workflow:** Request Access -> Approval -> Access granted for 2 hours -> Access Revoked.
- Case Study: [Microsoft Azure PIM \(Privileged Identity Management\)](#)
 - Usage: Admins are "Eligible" for a role, but do not hold it. They must "Activate" it to use it.

Why does your Domain Admin need Domain Admin rights at 3:00 AM on a Sunday? They don't. But if their account is compromised at 3:00 AM, the attacker inherits those rights.

We are moving to 'Zero Standing Privileges' via Just-in-Time (JIT) access. In this model, an admin is not an admin 24/7. They are 'eligible' to become an admin. When they need to do work, they request access. They might need to solve an MFA challenge or get manager approval. Access is granted for a specific window, say, two hours.

Once the window closes, the privileges are stripped. This narrows the window of opportunity for an attacker. Even if they compromise the credential, it has no power until it goes through the activation workflow. Azure PIM is the standard implementation of this in the Microsoft ecosystem.



Public Key Infrastructure (PKI) & Machine Identity

- The Trust Anchor:
 - **Role:** Managing digital certificates for Users, Devices, and Services.
 - **Challenge:** Short-lived certificates for cloud workloads (TTL = 1 hour).
- **Automation:**
 - [ACME Protocol](#): Automated certificate renewal, remove manual key handling
 - [SPIFFE](#): Provides standardized identity for workloads, Enables secure service-to-service authentication

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Public Key Infrastructure is not new, but its role has fundamentally expanded. In traditional environments, PKI was often associated with HTTPS certificates and occasional device authentication. In modern cloud-native systems, PKI underpins nearly every trust decision.

The idea of a trust anchor is critical. PKI allows us to bind an identity, whether human or machine, to a cryptographic key. That binding is what allows systems to authenticate each other without relying on IP addresses, network zones, or shared secrets. This becomes essential in environments where workloads are ephemeral and network locations are meaningless.

Machine identity is where this really becomes operationally challenging. Services now communicate with other services constantly, often without human involvement. These interactions must be authenticated, authorized, and auditable. Static credentials simply do not scale and represent a significant risk. This is why automation is not optional. Protocols like ACME and identity frameworks like SPIFFE exist to remove humans from the certificate lifecycle as much as possible.

From a security perspective, short-lived certificates are a feature, not a problem.

They reduce the window of abuse if a credential is compromised and force systems to re-authenticate frequently. When combined with policy enforcement, this allows Zero Trust principles to operate continuously rather than as a one-time check.

The key insight to remember is this: in modern architectures, identity is inseparable from cryptography. If you cannot issue, rotate, validate, and revoke certificates reliably, you cannot claim to have Zero Trust. Certificates become the passport that workloads and users present on every interaction, and PKI is the system that decides whether that passport is valid.



Policy and Access Enforcement



Policy as Enforcement Fabric

- Decoupling Policy from Code:
 - Problem: Hardcoded security logic (if user == admin) is brittle and hard to audit.
 - Solution: Policy-as-Code (OPA - Open Policy Agent).
- Example: "Container images must come from Trusted Registry X."
- Case Study: [Stripe's Security Architecture](#)
 - Method: **Enforcing invariants at the infrastructure level.**
 - Benefit: Developers cannot accidentally deploy insecure infrastructure because the Policy Engine rejects the deployment.

How do we enforce security at scale? Do we ask developers nicely to check for vulnerabilities? No. We use 'Policy as Code.' We decouple the decision-making logic from the application logic.

Tools like OPA (Open Policy Agent) allow us to write rules, policies, that are enforced by the infrastructure. For example, we can write a policy that says: 'No Load Balancer shall be exposed to the public internet unless it has the tag Public=True.' If a developer tries to deploy a load balancer without that tag, the deployment fails.

This is what we call an 'Enforcement Fabric.' It catches misconfigurations before they become vulnerabilities. Stripe is famous for this approach. They treat security invariants as code. This prevents attacks as the attacker or the negligent insider cannot disable the controls; the controls are baked into the deployment pipeline itself. We are moving from 'Detecting' bad behavior to 'Rejecting' it



Network Security as Risk Management

- Adaptive Security :
 - Stallings: FIPS 199 (Static Categorization).
 - Modern: **Continuous Adaptive Risk and Trust Assessment** (CARTA)
 - Risk signals may include: **Identity, Device posture, Location, Time and behavior**
 - Controls adjust dynamically rather than remaining fixed
- Scenario: User logs in from New York (Risk: Low). 5 mins later logs in from Moscow (Risk: Critical). [T1078]
- Case Study: [Okta Risk-Based Authentication](#)
 - Mechanism: Assigns a risk score to every login event based on device, location, and velocity.
 - Action: If Risk > Medium, prompt for MFA. If Risk > High, Deny.

Stallings introduces us to FIPS 199, where we categorize systems as Low, Moderate, or High impact. That's a great starting point, but it's static. In modern operations, risk changes by the second.

We are adopting a model called Continuous Adaptive Risk and Trust Assessment (CARTA). This treats network security as a real-time risk management problem. Imagine a user logging in. They have the right password. A traditional firewall lets them in. A Risk-Based system asks: 'They are logging in from New York. Five minutes ago, they logged in from London. That is physically impossible.' The system assigns a high risk score to that session and dynamically steps up enforcement, perhaps demanding a FIDO2 hardware key or blocking the request entirely. Okta and other identity providers prioritize this. It enables us to detect such attacks because while the credentials are valid, the *context* is malicious.



Module 3 : Micro-Segmentation



Micro-segmentation: The New VLAN

- Blast Radius Reduction:
 - Macro-segmentation: VLANs (HR, Engineering, Servers). Too broad.
 - **Micro-segmentation:** Host-to-Host or Workload-to-Workload rules.
- Rule: "Web Server A can talk to App Server B on port 443. Nothing else."
- Case Study: [Maersk \(Post-NotPetya\)](#)
 - Correction: After the flat network allowed NotPetya to destroy 45,000 PCs, Maersk implemented rigorous segmentation.
 - Goal: Prevent lateral movement (TA0008).

We talked about the failure of the 'Trusted VLAN' previously. The solution is Micro-segmentation. VLANs are no longer enough. Putting all your servers in a 'Server VLAN' just means that if one server is breached, they are all breached. Micro-segmentation applies Zero Trust principles to internal traffic. We define rules at the workload level: 'This specific Web Server can talk to this specific Database on this specific port, and nothing else.' Even if they are on the same subnet, they cannot ping each other.

After Maersk was flattened by NotPetya, this was their primary remediation. They moved from a flat global network to a highly segmented architecture. This is designed to limit the 'Blast Radius.' If an attacker compromises one host, they are trapped in a cell; they do not have the run of the prison. This directly counters Lateral Movement techniques like T1021 SSH.



The Modern DMZ

- VPC Design Patterns:
 - Old DMZ: Physical firewalls sandwiching web servers.
 - **Cloud DMZ**: Public Subnets vs. Private Subnets.
- Rule: **Only Load Balancers and NAT Gateways go in Public. Apps and Data go in Private.**
- Case Study: [AWS VPC Design](#)
 - Pattern: The "Three-Tier Architecture" (Web, App, DB) adapted for Cloud.
 - Control: Security Groups (Stateful) + NACLs (Stateless) acting as layered defense.

- If your database has a public IP, you have already failed.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The DMZ (Demilitarized Zone) is not dead; it just evolved. In the cloud, we build DMZs using VPC (Virtual Private Cloud) design. We distinguish between Public Subnets and Private Subnets.

A Public Subnet has a route to the internet gateway. A Private Subnet does not. The Golden Rule of Modern Network Security is: Your Application and your Database live in the Private Subnet. Only the Load Balancer lives in the Public Subnet.

This means that even if you forget to patch your database, it is not reachable from the internet. There is no route. Attackers must first compromise the Load Balancer or the Bastion Host to reach the data. This layering of network reachability is fundamental to reducing the attack surface.



Hybrid Connectivity: Direct Connect & ExpressRoute

- The Physical Bridge:
 - Concept: Dedicated fiber links between On-Prem and Cloud. Bypasses the public internet.
- Risk: The "**Bypass**" effect. Firewalls often trust this traffic implicitly.
- Case Study: [Ivanti VPN Exploitation \(2024\)](#)
 - Control:
 - MACsec Encryption: Encrypting the physical layer (L2) to prevent wiretapping.
 - **Transit Gateway**: Centralizing the connection to inspect traffic before it fans out to VPCs.
 - Place a **Virtual Firewall** at the cloud entry point.

- A private link is just a private tunnel for malware.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

When large companies connect to the cloud, they don't use the internet; they use dedicated lines like AWS Direct Connect or Azure ExpressRoute. These are physical fibers.

The mistake architects make is trusting this link. 'It's private, so it's safe.' Wrong. If your on-prem network is infected, that private link is the superhighway for the malware to reach the cloud. You must encrypt this link (MACsec) and, more importantly, you must terminate it at a Transit Gateway with a firewall. Inspect the traffic coming from your own data center as if it were hostile.



Module 4 : Workload Security



The API Gateway as the New Perimeter

- Gatekeeper for Microservices:
 - Role: Authentication, Rate Limiting, Input Validation, Telemetry.
 - Stallings: "Complete Mediation" (Checking every access).
- Case Study: [T-Mobile API Breach \(2023\)](#)
 - Failure: No rate limiting on a customer service API.
 - Attack: Scrapers downloaded 37 million records by iterating through IDs [T1005]
 - Solution: An API Gateway enforcing strict rate limits per user.

If the API is the front door, the API Gateway is the bouncer. In a microservices architecture, you cannot rely on every individual service to handle security perfectly. You centralize it at the Gateway.

The Gateway handles Authentication, Rate Limiting, and Input Validation. It ensures 'Complete Mediation,' one of Stallings' key design principles. T-Mobile failed this in 2023. They had an API that was technically authenticated, but it had no rate limits. Attackers queried it millions of times, scraping 37 million records. An effective API Gateway would have seen this anomaly, 'Why is this one user requesting 10,000 records a minute?', and shut it down. The Gateway is your primary defense against automated scraping and data exfiltration via API.



Runtime Application Self-Protection

- Layer 7 Defense:
 - WAF: Inspects incoming HTTP traffic for SQLi, XSS, RCE. (Perimeter).
 - RASP (**Runtime Application Self-Protection**): Runs inside the app code. (Interior).
- The Difference:
 - WAF guesses if traffic is bad based on signatures.
 - RASP sees the app executing the SQL query and blocks it if it's malicious.
- Case Study: [Log4j Defense](#)
 - WAF: Blocked strings like \$jndi:.
 - RASP: Blocked the Java process from making the socket connection.

- Defense in depth means protecting the code from the inside out.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

WAFs are the first line of defense for web apps. They filter out the noise, the script kiddies and scanners. But they can be bypassed with obfuscation. That's why we need RASP (Runtime Application Self-Protection). RASP is a library loaded by your application. It hooks into the runtime (like the JVM). If an attacker manages to bypass the WAF with a SQL injection, the RASP sees the actual SQL query being constructed. It sees DROP TABLE and kills the request. It works even when the WAF fails.



Service Mesh (mTLS & Observability)

- **The East-West Traffic Cop:**
- Problem: In microservices, thousands of services talk to each other. Firewalls are too slow/coarse.
 - Solution: Service Mesh (Istio, Linkerd).
- Capability: **Mutual TLS** (mTLS):
- Function: Automatically encrypting traffic between services and verifying identity (Certificate-based).
- Result: Zero Trust inside the cluster.
- Case Study: [Monzo Bank](#)
- Architecture: Uses Service Mesh to enforce strict isolation between banking microservices (e.g. 'Ledger' cannot talk to 'Frontend').

- Encryption in transit, by default, everywhere.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Service Mesh is the advanced class of segmentation. When you have Kubernetes running 5,000 pods, you can't use a firewall to segment them. Service Mesh solves this by injecting a tiny proxy (Sidecar) next to every container. This proxy handles encryption. It creates a Mutual TLS (mTLS) tunnel between Service A and Service B. This means (1) the traffic is encrypted, and (2) they cryptographically authenticate each other. If an attacker is on the network sniffing packets, they see nothing. If they try to impersonate a service without the certificate, they get rejected. It solves encryption and identity at scale without the developer writing code.



Implementing mTLS

- Abstracting Security from Code:
 - Architecture: Every application container is paired with a "**Sidecar**" proxy (Envoy).
 - Function: The Sidecar handles mTLS, logging, and routing. The App knows nothing about it.
 - Benefit: Consistent security posture across polyglot services (Java, Go, Node).
- Case Study: [Istio Security](#)
 - Feature: Automatic mTLS.
 - Result: All traffic is encrypted in transit, and services are mutually authenticated, without developers writing a single line of crypto code.

- Developers are bad at cryptography. Take it away from them.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

We know developers are not cryptographers. If you ask 10 developers to implement TLS, you will get 10 different implementations, and 5 of them will be insecure. The 'Sidecar' pattern solves this.

In a Service Mesh, we attach a small proxy container, a Sidecar, to every application container. All network traffic goes through the Sidecar. The Sidecar handles the mutual TLS or mTLS handshake, the certificate rotation, and the logging. The application just talks to 'localhost.'

This abstracts security out of the application layer and into the infrastructure layer. It ensures that even if a developer forgets to enable HTTPS, the mesh enforces it. This effectively kills Network Sniffing inside your data center because every single packet is encrypted by default.



Module 5 : Runtime Security



Configuration Drift

Configuration drift occurs when systems gradually deviate from their **intended, approved, or secure configuration state** over time.

Why Drift Happens

- Manual changes during troubleshooting or incidents
- Emergency fixes not reverted
- Inconsistent deployment across environments
- Cloud scale and automation without guardrails

Why Drift Is Dangerous

- Security controls silently weaken
- Compliance posture degrades without alerts
- Creates exploitable gaps without new vulnerabilities

Where Drift Commonly Appears

- Network access rules and routing policies
- Cloud security group and firewall configurations
- Identity and access permissions
- Logging and monitoring settings

Managing Configuration Drift

- Define a secure baseline
- Continuously monitor configuration state
- Detect deviations early
- Enforce remediation through automation

Let's talk about one of the most common causes of real-world security failures, and one that almost never makes headlines on its own: configuration drift. Configuration drift is not a hack, it's not malware, and it's not an exploit. It's what happens when a system that was once correctly configured slowly, quietly becomes something else.

Drift happens because environments change. Engineers troubleshoot issues and make temporary exceptions. Incidents force emergency changes. Teams deploy new services faster than policies are updated. In cloud environments especially, scale and automation mean that small inconsistencies multiply quickly. None of these actions are malicious, and that's precisely why drift is so dangerous. There is no alarm when it happens.

From a security perspective, configuration drift undermines assumptions. You may believe a network is segmented, but one overly permissive rule breaks isolation. You may believe logging is enabled, but a disabled setting removes visibility. You may believe access is tightly controlled, but accumulated permissions violate least privilege. No new vulnerability needs to be introduced. The system becomes vulnerable simply because it no longer matches its intended state.

Industry frameworks explicitly recognise this risk. Secure configuration

management is a foundational control because security is not a one-time setup. It is a continuous process. This is why modern security programs emphasize baselines, continuous monitoring, and automated enforcement rather than manual reviews. In cloud and hybrid environments, this often means treating configuration as code and continuously validating that reality matches policy. The key insight here is that many breaches do not begin with sophisticated attacks. They begin with assumptions that are no longer true. Configuration drift is the silent failure mode that turns good architecture into weak security over time. If you are not actively detecting and correcting drift, you are not maintaining security, you are hoping it stays intact.



Cloud Security Posture Management (CSPM)

- The "Config" Watchdog:
 - Problem: Cloud APIs allow thousands of config changes per hour. Drift is inevitable.
 - Solution: CSPM tools (Wiz, Prisma Cloud, Orca).
- Function: Continuously scanning APIs for misconfigurations against a baseline (CIS Benchmarks).
- Case Study: [AWS S3 Leaks \(BlueBleed\)](#)
- Detection: CSPM tools would detect "Public Read" on a blob storage container instantly and trigger an alert (or auto-remediate).

- You cannot audit the cloud manually.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In the cloud, your infrastructure is defined by API settings. A single developer toggling a switch can expose the entire company. You cannot audit this with a clipboard.

CSPM (Cloud Security Posture Management) is the answer. These tools hook into the Cloud APIs and read the configuration of every resource, every S3 bucket, every Security Group, every IAM role. They compare it against a policy (like CIS Benchmarks). If a bucket is made public, the CSPM sees it in seconds. Mature organizations turn on 'Auto-Remediation,' where the CSPM bot automatically reverts the bucket to private. It's a self-healing security posture.



Cloud Workload Protection (CWPP)

- The "Runtime" Defense:
 - Focus: Securing the compute (VMs, Containers, Serverless).
 - Capabilities: Vulnerability Scanning, Runtime Defense, File Integrity Monitoring (FIM).
 - Shift: Moving from "Anti-Virus" (File scanning) to "Behavioral Analysis" (Process anomalies).
- Case Study: [Siloscape Malware](#)
 - Detection: CWPP detects the behavior of a container trying to escape to the host node (syscall anomalies).

- Protecting the workload, wherever it runs.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

CSPM handles the configuration; CWPP (Cloud Workload Protection Platform) handles the runtime. This is the evolution of Antivirus. In a container, files are ephemeral, so traditional AV doesn't work well.

CWPP sits inside the OS or on the Hypervisor. It watches the behavior. If a web server process (nginx) suddenly spawns a shell (bash), that is an anomaly. If a container tries to mount the host file system, that is an escape attempt. CWPP blocks these behaviors in real-time. It is essential for protecting the 'Compute' layer of your cloud stack.



Secrets Management

- The Problem of Secrets Sprawl:
 - Issue: Hardcoded API keys, Secrets copied across environments and repositories, database passwords, and certs in code, Long-lived credentials with no rotation
 - Solution: Centralized Vaults (HashiCorp Vault, AWS Secrets Manager).
- Mechanism: App authenticates to Vault -> Vault returns temporary credential.
- Case Study: [Samsung Source Code Leak \(LAPSUS\\$\)](#)
 - Incident: 190GB of source code stolen.
 - Findings: Hardcoded cryptographic keys and credentials found throughout the codebase. [T1552.001]
- Case Study: [Uber \(2022\) Redux](#)
 - Failure: Hardcoded PAM admin credentials in a script. [T1552]
 - Fix: If the script had called a Vault, the attackers would have found code, not keys.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

One of the most consistent failure patterns in modern breaches is secrets sprawl. Secrets are exactly what the name implies: anything that proves identity or grants access. API keys, database passwords, certificates, tokens. In theory, these are tightly controlled. In practice, they end up hardcoded in applications, embedded in scripts, copied between environments, and forgotten.

The reason this is so dangerous is simple. Secrets bypass many other security controls. If an attacker has a valid key, they don't need to exploit a vulnerability, bypass a firewall, or evade detection. They authenticate. From a network security perspective, secrets collapse trust boundaries. What looks like legitimate traffic is, in fact, attacker-controlled access.

The architectural fix is not "be more careful with secrets." That does not scale. The fix is to centralize secrets management and remove secrets from code entirely. Applications authenticate to a trusted service using their own identity, and that service returns temporary credentials. Those credentials expire automatically, are scoped to exactly what is needed, and can be revoked centrally. This shifts secrets from static artifacts into controlled, auditable workflows.

The real-world incidents listed here reinforce this point. In the Samsung source code leak, attackers found cryptographic keys embedded throughout the

codebase. In the Uber breach, hardcoded privileged credentials in a script gave attackers a foothold. In both cases, the problem was not a zero-day vulnerability. The problem was that secrets were treated as configuration instead of as high-risk assets.

The key takeaway is this: source code exposure should be embarrassing, not catastrophic. If reading your code gives an attacker credentials that unlock systems, then secrets management has failed. Modern network security assumes that code will leak eventually. The goal is to ensure that when it does, attackers find logic, not keys, and permissions that expire before they can be abused.



Immutable Infrastructure

- Cattle vs. Pets:
 - Pets: Servers with names (e.g. "Gandalf"), manually patched, nursed back to health.
 - Cattle: Ephemeral instances, replaced/terminated, never patched.
- Security Benefit: Persistence is impossible.
- Case Study: [Capital One's Rehydration Policy](#)
 - Strategy: Servers are terminated and replaced every 24-48 hours.
 - Impact: An attacker who compromises a box loses access automatically when the box is killed.

- Don't patch a compromised server; shoot it.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In the old days, we treated servers like pets. We gave them names like 'Gandalf' or 'Zeus.' When they got sick (hacked or buggy), we logged in and tried to fix them. Modern security demands 'Immutable Infrastructure.' We treat servers like cattle. We don't patch them; we replace them. If a new vulnerability comes out, we build a new image, deploy 100 new servers, and terminate the 100 old ones. This has a profound security benefit: it kills Persistence. Attackers spend days establishing a foothold, installing rootkits, setting up scheduled tasks. In an immutable environment, the server they hacked is scheduled to die in 24 hours. Their foothold is wiped out automatically. 'Rehydration', cycling your infrastructure constantly, is one of the most effective anti-APT strategies we have.



Bastion Hosts vs. Session Manager

- Secure Administration:
 - Legacy: Jump Box / Bastion Host with Public IP and SSH (Port 22).
 - Modern: SSM Session Manager (AWS) / Azure Bastion.
- The Shift:
 - No Open Ports: Uses agent-based outbound connectivity.
 - Identity Auth: Login via IAM, not SSH keys.
 - Full Recording: Every keystroke logged to S3/CloudWatch.
- Case Study: [Removing SSH Keys](#)
- Action: Disabling Port 22 on all Security Groups.
- Result: Elimination of brute-force attacks and key management headaches.

- Port 22 is a relic. Close it.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Stop building Jump Boxes. Stop opening Port 22. Managing SSH keys is a nightmare, and public-facing SSH servers are constantly brute-forced. The modern standard is tools like AWS Systems Manager (SSM) Session Manager. It installs an agent on the server. The agent dials *out* to the AWS cloud. You authenticate to AWS via IAM (MFA supported), and AWS tunnels your shell session down to the agent. No inbound ports. No SSH keys to lose. And best of all, it records every single keystroke to a log file for audit. It is safer, easier, and compliant.



Module 6 : Internet-Dependent Control Planes



DNS Security: The Control Plane

- Protective DNS (PDNS):
 - Function: Blocking outbound resolution of known malicious domains (C2, Phishing).
 - Value: Stops the "Call Home" phase of the Kill Chain. [T1071.004]
- DNSSEC:
 - Function: Cryptographically signing DNS records to prevent spoofing/poisoning.
- Case Study: [NSA & CISA Mandate](#)
 - Action: Selecting a Protective DNS service is now mandatory for US Federal Agencies.
 - Why: It is the highest ROI control for stopping ransomware C2. [T1568]

- DNS is the phonebook of the internet; don't let malware look up numbers.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

DNS is often ignored, but it is a critical control plane. Almost every piece of malware needs to resolve a domain name to phone home to its Command and Control (C2) server.

If you control DNS, you break the Kill Chain. This is called Protective DNS (PDNS). Instead of resolving malware.com to an IP, the DNS server resolves it to a 'Sinkhole' IP. The malware thinks it connected, but it's talking to your security team.

The NSA mandated this for all federal agencies because it is high-impact and low-cost. We also look for DGA (Domain Generation Algorithms), malware that generates random domain names like xyz123.com. PDNS analyzes the entropy of these requests and blocks them automatically.



Secure Routing: BGP and RPKI

- The Internet's Trust Deficit:
 - BGP (Border Gateway Protocol): Built on trust. Any ASN can announce any IP prefix.
 - Threat: BGP Hijacking (intentional or accidental) reroutes traffic globally.
 - The Solution: RPKI (Resource Public Key Infrastructure):
- Mechanism: Cryptographically signing Route Origin Authorizations (ROAs).
- Action: Route filtering. "Drop Invalid."
- Case Study: [YouTube & Pakistan Telecom \(2008\)](#)
 - Incident: Pakistan tried to block YouTube locally via BGP.
 - Result: They accidentally announced the route to the world. Global traffic for YouTube went to Pakistan.

- Route filtering is the firewall for the internet.

BGP holds the internet together, and it is terrified of duct tape. It relies on trust. If I say 'I own Google's IP addresses,' the internet believes me. This allows for BGP Hijacking, redirecting traffic to steal data or deny service.

The fix is RPKI, Resource Public Key Infrastructure. It's like a digital signature for IP routes. As a network defender, you must sign your prefixes (ROA) and, critically, you must configure your routers to 'Drop Invalid' updates. This ensures that if someone tries to hijack your address space, the rest of the secure internet will ignore them. The YouTube/Pakistan incident proved how fragile BGP is; RPKI is how we fix it.



E-Mail Security (DMARC, SPF, DKIM)

- The Original Threat Vector: Phishing is still #1.
- The Standards:
 - SPF: "These IPs can send mail for me."
 - DKIM: "This mail is cryptographically signed by me."
 - DMARC: "If SPF/DKIM fail, delete the email."
- Case Study: Business Email Compromise (BEC)
- Defense: Enforcing DMARC 'Reject' policy prevents attackers from spoofing your CEO's domain to external partners.

- Authentication is not just for users; it's for domains too.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Email is 50 years old and insecure by design. Anyone can write 'From: ceo@company.com!'. To fix this, we use the trifecta: SPF, DKIM, and DMARC. SPF lists authorized IPs. DKIM signs the headers. But DMARC is the policy enforcement. It tells the world: 'If you receive an email claiming to be from me, and the signature is bad, burn it.' Implementing DMARC at 'Reject' level is the single most effective way to stop your domain from being used in fraud.



Module 7 : Visibility



Visibility as Prerequisite

- The Observability Challenge:
 - Problem: Ephemeral containers live for seconds. Standard logs miss them.
 - Solution: eBPF (Extended Berkeley Packet Filter).
- Capability: Sandboxed programs running in the Linux Kernel to observe all system calls and network packets without changing app code.
- Case Study: [Cilium Project](#)
- Usage: Providing L3-L7 visibility and security for Kubernetes using eBPF.
- Value: Seeing every DNS request and API call, even between containers on the same host.

- You cannot secure what you cannot see. In Cloud, seeing requires kernel-level depth.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

There is a military maxim: 'You cannot hit what you cannot see.' In the cloud, visibility is our biggest challenge. A Kubernetes pod might exist for 30 seconds to process a job and then vanish. If you are polling for logs every 5 minutes, you missed it entirely. The attacker came and went in the gap.

To solve this, we are turning to eBPF, Extended Berkeley Packet Filter. This is a technology that allows us to run sandboxed programs directly inside the Linux kernel. It gives us 'God Mode' visibility. We can see every network packet, every file access, and every process execution, without modifying the application code.

Projects like Cilium use eBPF to provide deep security observability for Kubernetes. We can see exactly which pod talked to which API, even if they are on the same physical server. This level of visibility is the prerequisite for any modern security program. If you are still relying on SNMP traps and ping checks, you are flying blind.



Detection-Driven Design

- Shift Left for Detection:
 - Legacy: Build network -> Add Logs -> Hope SIEM catches it.
 - Modern: Design the network to be detected.
 - Concept: Detection as Code (Sigma Rules).
- Case Study Revist: [SolarWinds Orion \(Sunburst\)](#) (2020)
- The Detection: FireEye detected the breach not via AV, but because an employee registered a second MFA device.
- Lesson: High-fidelity, low-volume alerts (Identity anomalies) beat high-volume network noise.

- If you can't detect it, you can't secure it.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

We used to treat logging as an afterthought. 'Turn on logging and send it to Splunk.' That creates a data swamp, not a detection capability.

Modern security uses 'Detection-Driven Design.' When we build a new VPC or deploy a new app, we ask: 'What log will this generate if it is attacked?' We write the detection rule (using Sigma or YARA) at the same time we deploy the infrastructure.

Look at how SolarWinds was found. It wasn't found by AI. It was found because FireEye had a specific alert for 'User Registering a New MFA Device.' It's a boring event, but usually a rare one. Because they noticed that anomaly, they unraveled the biggest supply chain attack in history. Design your network to generate high-signal logs.



Network Detection and Response (NDR)

- The Eye on the Wire:
 - Role: analyzing raw traffic (East-West) for anomalies using ML.
 - Why: Logs can be disabled; the network never lies.
- Capabilities:
 - Behavioral Analysis: "This printer is doing a port scan."
 - Decryption: Analyzing decrypted traffic streams.
- Case Study: DarkSide Ransomware
- Detection: NDR tools detected the massive SMB data transfer (Exfiltration) before encryption started.

- Logs tell you what the system said it did; NDR tells you what it actually did.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Logs are great, but logs can be turned off. Attackers often kill the logging agent first. But they cannot turn off the network. If they want to steal data, packets must move.

NDR (Network Detection and Response) sits on the wire (via span port or VPC mirror). It uses Machine Learning to learn 'normal.' It knows that a Printer talks to the Print Server. If the Printer starts scanning the Domain Controller, the NDR alerts immediately. It is the ultimate truth source in the network.



Security Orchestration Automation and Response

- The Brain of the SOC:
 - SIEM (Security Information and Event Management): The Aggregator. "Here is a pile of logs." (Splunk, Sentinel).
 - SOAR (Security Orchestration, Automation, and Response): The Actor. "Do something about it." (XSOAR, Tines).
- The Workflow:
 - SIEM detects "Brute Force SSH." -> alerts SOAR.
 - SOAR runs playbook: "Block IP on Firewall" + "Disable User in AD."
- Case Study: Reducing MTTR (Mean Time to Respond)
- Metric: Moving response time from 4 hours (Human) to 4 seconds (Bot).

You cannot scale a SOC with humans. You need automation. SIEM is the place where logs go to live (or die). It provides visibility. But SOAR is where action happens.

If your SIEM sees a phishing email, a human analyst has to check it, delete it, reset the user's password, and block the sender. That takes 30 minutes. A SOAR playbook does that in 3 seconds. Modern architecture demands SOAR to handle the volume of alerts generated by a Zero Trust environment.



Module 8 : Edge Controls



User & Data Egress Controls

Most modern breaches succeed not at entry, but at **data exfiltration** and **command-and-control over outbound traffic**.

What Are Egress Controls?

- Controls that **govern, inspect, and constrain outbound user traffic and data flows** from endpoints to the Internet and cloud services.

Primary Objectives

- Prevent data leakage and unauthorized uploads
- Detect malicious outbound connections
- Enforce acceptable use and security policy
- Reduce exposure to web-based threats

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Traditional security thinking focused heavily on ingress: firewalls, VPNs, and perimeter defenses designed to keep attackers out. But modern incidents consistently show that attackers often get in through valid credentials, trusted devices, or supply chain paths. The real damage happens later, when data leaves the organization or when compromised systems communicate outward.

This is why egress controls matter. Egress controls focus on outbound traffic, particularly user-initiated traffic to the web and cloud services. They are about governing what users can access, what data can leave, and how external content interacts with internal systems. From a network security perspective, outbound traffic is often more revealing than inbound traffic, because it reflects intent, misuse, and compromise.

The three control categories we're introducing here each address this problem from a different angle. Secure Web Gateways focus on inspecting and enforcing policy on web traffic. Cloud Access Security Brokers focus on visibility and control over cloud services, especially sanctioned versus unsanctioned use. Remote Browser Isolation takes a more radical approach by assuming the web is hostile and executing it away from the endpoint entirely.

What's important is not to treat these as standalone tools. They are architectural responses to the same problem: users need access to the Internet and cloud

services, but that access must be mediated, observed, and constrained. These controls sit logically between the user and external services, forming a distributed, user-centric perimeter.

The key takeaway is this: if your security strategy only focuses on preventing entry, you are assuming perfect prevention. Modern network security assumes compromise and focuses on limiting impact. Egress controls are how organizations prevent compromised users, devices, or sessions from turning access into data loss or persistent control. In the next slides, we'll break each of these control types down in detail, but this framing should help you understand why they exist and how they fit together.



Secure Access Service Edge (SASE)

- The Convergence:
 - Definition: Combining WAN capabilities (SD-WAN) with Network Security services (SWG, CASB, FWaaS) into a single cloud model.
 - Driver: The user is off-network; the data is off-network. Why backhaul to HQ?
- Core Components:
 - SWG: Secure Web Gateway (Web filtering/Malware).
 - CASB: Cloud Access Security Broker (SaaS visibility).
 - ZTNA: Zero Trust Network Access (Private app access).
- Case Study: [NVIDIA's Move to SASE](#)
- Shift: Moved from 50 VPN concentrators to a global SASE cloud. Reduced latency and improved security posture.

- The data center is no longer the center of data.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

SASE (Secure Access Service Edge) is the architecture of the remote workforce. In the old days, if you were at home and wanted to browse the internet, your VPN forced your traffic all the way to Headquarters, through the corporate firewall, and then out to the web. This is slow and expensive (Tromboning). SASE puts the security stack in the cloud, at the 'Edge.' You connect to the nearest POP. That POP scrubs the traffic (SWG), checks your SaaS permissions (CASB), and connects you to private apps (ZTNA). It is a unified security model that follows the user, rather than forcing the user to come to the security.



Secure Web Gateway (SWG)

- The Internet Filter:
 - Legacy: On-prem Proxy servers (BlueCoat).
 - Modern: Cloud-native SWG (Zscaler, Netskope).
- Critical Feature: TLS Inspection:
 - Reality: 90% of malware is encrypted. SWG must decrypt, scan, and re-encrypt.
- Case Study: Blocking Typosquatting
- Scenario: User types goggle.com instead of google.com.
- Action: SWG categorizes the domain as "Suspicious/New" and blocks the connection before the exploit kit loads.

- The browser is the most dangerous application in the enterprise.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The Secure Web Gateway (SWG) is the bodyguard for your users when they browse the internet. It replaces the old on-prem proxy. Critically, it handles TLS Inspection at scale.

Since malware is encrypted, the SWG acts as a Man-in-the-Middle (legitimately). It decrypts the traffic, scans it for malware signatures, and then re-encrypts it to send it to the user. It also enforces policy: 'No Gambling sites,' 'No Malware sites,' and 'No newly registered domains.' That last one is crucial, blocking domains that are less than 30 days old stops a huge percentage of phishing attacks.



Cloud Access Security Broker (CASB)

- The Shadow IT Controller:
 - Problem: Employees using unapproved SaaS (Dropbox, WeTransfer) to move corporate data.
- Solution: CASB.
- Deployment Modes:
 - Forward Proxy: Intercepts traffic at the edge to block uploads to personal accounts.
 - API Mode: Scans data already in the cloud for sensitive info (DLP).

- You can't stop SaaS, but you can govern it.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

"Employees want to do their jobs. If IT is slow, they will use Dropbox or Google Drive. This is Shadow IT. A CASB (Cloud Access Security Broker) is how we tame this.

A CASB sits between your users and the cloud. It can see that a user is logging into *Corporate* Gmail vs. *Personal* Gmail. It can allow the corporate login and block the personal one. It can also scan files being uploaded for credit card numbers (DLP) and block the transfer. Without a CASB, your data is leaking out of a thousand SaaS pinholes.



Remote Browser Isolation (RBI)

- Containing the Threat:
 - Concept: Executing web code in a remote disposable container, streaming only pixels to the user.
 - Benefit: Malicious JS/Exploits never reach the endpoint.
 - Target: High-risk users (HR opening resumes, Finance opening invoices).
- Case Study: Watering Hole Attacks
- Defense: RBI renders the infected site in the cloud. The exploit fires in a disposable container, user stays safe.

The web browser is executing untrusted code from the internet all day long. Remote Browser Isolation (RBI) accepts this risk and moves it off the endpoint. When a user visits a risky site, the site loads on a server in the cloud. That server turns the webpage into a video stream (pixels) and sends it to the user. The user can click and scroll, but no code runs on their machine. If the site has a zero-day exploit, it crashes the cloud container, not the CEO's laptop.



Data Loss Prevention (DLP)

- The Final Check:
 - Goal: Detect/Block sensitive data (PII, PCI, IP) leaving the environment.
 - Locations: Endpoint (USB), Network (Uploads), Cloud (API).
- Techniques:
 - Regex: Pattern matching (CC numbers).
 - Exact Data Match (EDM): Fingerprinting specific DB rows.
 - Optical Character Recognition (OCR): Reading text in images.

We secure the network to secure the data. DLP (Data Loss Prevention) is the control that looks at the data itself. It's hard to get right because it generates noise.

Modern DLP goes beyond Regex. It uses Exact Data Match (EDM) to fingerprint your actual customer database. It uses OCR to read text inside images, because attackers will screenshot a document to bypass a text filter. It runs on the endpoint to stop USB transfers and on the CASB to stop cloud uploads.



Module 9 : Resilience



DDoS Mitigation: Layer 3 vs. Layer 7

- The Availability Defense:
 - Volumetric (L3/L4): Flooding with UDP/SYN packets.
 - Defense: Anycast routing, Scrubbing centers (Cloudflare, Akamai).
- Application (L7): Resource exhaustion (Search queries, Login attempts).
 - Defense: Rate limiting, CAPTCHA, Behavioral analysis.
- Case Study: [GitHub \(2018\)](#) vs. [Google \(2023\)](#)
 - GitHub: Memcached amplification (1.3 Tbps). Stopped by filtering UDP.
 - Google: HTTP/2 Rapid Reset (398 Million RPS). Stopped by protocol patching and edge filtering.

- Scale is the only defense against scale.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

You cannot fight a DDoS attack with a firewall in your basement. Your internet pipe will fill up before the firewall even sees the traffic. You need cloud scale. For Volumetric attacks (Layer 3/4), we use Anycast. We spread the traffic across global networks like Cloudflare or Akamai. They absorb the tsunami. For Application attacks (Layer 7), like the HTTP/2 Rapid Reset, we need smarter logic. We need rate limiting and behavioral challenges (CAPTCHAs). The goal is to make the attack expensive for the adversary and cheap for us.



Backup & Recovery (The Last Line)

- Resilience Engineering:
 - Rule: 3-2-1 Backup Strategy (3 copies, 2 media, 1 offsite/offline).
 - Ransomware Defense: Immutable Backups (WORM - Write Once Read Many).
- Case Study: [Garmin Ransomware](#)
 - Analysis: Restoration was slow/impossible, forcing payment.
 - Contrast: Hydro Norsk (Restored from backups, did not pay).

- Backups are useless. Restores are priceless.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Security controls fail. When they do, and Ransomware hits, your survival depends on one thing: Backups. But attackers know this. They target the backups first.

You need 'Immutable Backups.' This is storage that cannot be deleted or overwritten, even by the root admin, for a set period (e.g 30 days). AWS S3 Object Lock does this. If you have immutable backups, you can refuse to pay the ransom. If you don't, you are at their mercy.



Module 10 : Continuous Security



Software Bill of Materials (SBOM)

- Software Bill of Materials:
 - Definition: A nested inventory of all ingredients (libraries) in software.
 - Driver: Executive Order 14028 (Post-SolarWinds).
- Usage:
 - Vulnerability Management: "Where are we running Log4j?" (SBOM query).
 - License Compliance: "Are we using GPL code?"
- Case Study: Log4j Response
- Without SBOM: Scanning every server for weeks.
- With SBOM: Querying the inventory database -> Results in seconds.

- You can't patch what you don't know you have.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

When Log4j hit, every CISO asked: 'Where are we vulnerable?' Most had no idea. They had to scan thousands of servers.

Companies with an SBOM (Software Bill of Materials) answered that question in 5 minutes. An SBOM is an ingredients list. 'This app contains Spring Boot, which contains Log4j version 2.14.' By indexing this, vulnerability management becomes a database query, not a scavenger hunt. This is the future of supply chain defense.

The SolarWinds attack showed us that the build system itself is a target. We must implement 'Supply Chain Security.' This means scanning our dependencies (using tools like OWASP Dependency Check) and generating an **SBOM** so we know exactly what is running in our environment. If a new vulnerability drops, we grep the SBOM, not the servers. We also need to scan for hardcoded credentials. Tools like git-secrets and trufflehog help us prevent API keys from leaking into GitHub, which was a vector in the CircleCI breach mentioned earlier.



Attack Surface Management (ASM)

- The Outside-In View:
 - Concept: Continuous discovery of external assets (Shadow IT, forgotten subdomains).
 - Tooling: Shodan, Censys, proprietary ASM scanners.
- Goal: Find the vulnerability before the attacker does.

- You can't protect what you don't know you own.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

our CMDB (Configuration Management Database) is lying to you. It lists what you *think* you have. ASM (Attack Surface Management) tells you what you *actually* have.

ASM tools scan the internet like an attacker would. They look for your domains, your IP ranges, your SSL certificates. They find the marketing website launched by an intern. They find the test server running RDP. ASM gives you the 'Attacker's Eye View' of your perimeter.



Deception Technology (Active Defense)

- Turning the Tables:
 - Concept: Planting fake assets (Honeytokens, Honeypots) to detect intruders.
 - The Trap: No legitimate user should ever touch a honeytoken. 100% Fidelity Alert.
- Examples:
 - Fake AWS Keys: Placed in code repos. If used, alert immediately.
 - Fake AD Credentials: In LSASS memory.
- Case Study: [Canarytokens](#)
- Scenario: Attacker steals a "database password" from a config file. When they try to use it, it triggers an alert with their IP address.

- Make the attacker afraid to touch anything.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Deception is Active Defense. It flips the asymmetry. Usually, the defender has to be right 100% of the time. With Deception, the *attacker* has to be right 100% of the time. If they touch one fake file, one fake credential, they burn themselves. We use 'Honeytokens.' For example, I put a fake AWS Access Key in a GitHub repository. It has no permissions, but it has rigorous logging. If anyone tries to use that key, I know instantly that (A) I have a breach, and (B) who they are. It is the highest fidelity alert in the SOC because there are no false positives.



Class Discussion



Overview & Objectives

In this session, you will move beyond theoretical definitions of security and practice the art of Architecture Decomposition. You will step into the role of a Lead Security Architect tasked with analyzing three distinct environments: a traditional on-premise enterprise, a cloud-native startup, and a complex hybrid-cloud logistics firm.

Learning Objectives:

- Learn to identify "Trust Boundaries" where different technology stacks meet (e.g On-Prem vs. Cloud, Vendor vs. Corporate).
- Differentiate between attacks that happen on the **Data Plane** vs the **Control Plane**
- Propose specific controls to mitigate the identified risks



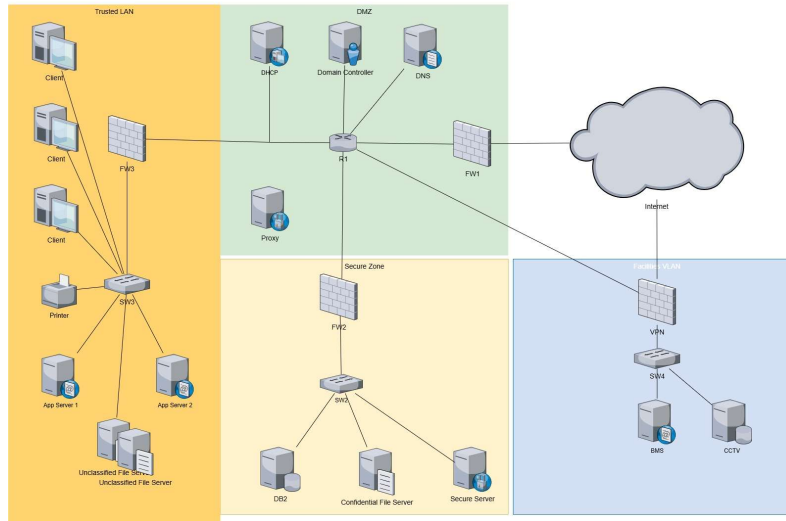
Exercise 1 : Traditional Enterprise ("LegacyBank Corp")

- LegacyBank is a regional financial institution. Their architecture is the classic "Castle and Moat" design.
- Perimeter: The network is protected by redundant Cisco ASA Firewalls and a dedicated DMZ for web banking.
- Internal Network: Behind the internal firewall lies a "Trusted" flat LAN where employees, printers, and internal application servers reside.
- Core Banking: The "Crown Jewels" the DB2 database holding customer balances, runs on a Mainframe in the most secure zone.
- Third-Party Access: A facilities management vendor (HVAC) has a permanent VPN tunnel into a segregated "Facilities VLAN" to monitor air conditioning units.



Network Diagram of LegacyBank Corp

LegacyBank Corp



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



The "Pivot" Attack

A threat actor has compromised the HVAC vendor's laptop via a phishing email. They have valid credentials to connect to the LegacyBank Facilities VPN. Answer the following questions:

- 1.1 How would the attacker verify connectivity upon connecting to the VPN? From least noisy to most noisy, map out possible tools/techniques the attacker could use, and the associated log/detection method which the blue team could have been alerted to it.
- 1.2 The Facilities VLAN is supposed to be isolated. How might an attacker jump to the Corporate VLAN? Similarly, what does the blue team need to monitor in order to detect this? Why do companies find it difficult to perform such detections?
- 1.3 Once on the Corporate LAN, how can an attacker move between Windows hosts? For each technique, list down the log/detection method.
- 1.4 You are the new CISO for LegacyBank, trying to get budget from the CEO to revamp this network. Perform a DREAD threat model for LegacyBank to convey your message.

- Email your answer to zongfu.chua@ntu.edu.sg with the Title "Week 3 Q1"

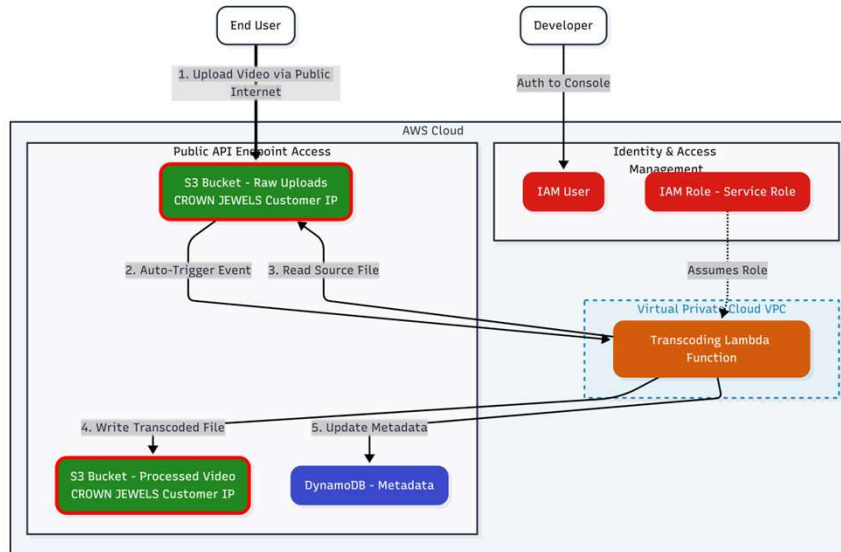


Exercise 2 : Cloud-Native Startup ("StreamFast")

- StreamFast is a video transcoding startup born in the cloud (AWS). They own no servers; everything is Serverless. Their cloud usage is as follow:
- Compute: AWS Lambda functions trigger automatically when users upload videos.
- Storage: S3 Buckets are used for raw uploads and processed video storage.
- Identity: Developers access the AWS Console using IAM Users. Production services use IAM Roles.
- Network: While they use a Virtual Private Cloud (VPC), most services (S3, DynamoDB) are accessed via public API endpoints.
- The "Crown Jewels": The S3 Buckets containing customer IP.



StreamFast Data Flow Diagram





The Management Plane Compromise

A developer accidentally committed their `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` to a public GitHub repository. These keys belong to a "PowerUser" account. Answer the following questions:

2.1 The attacker authenticates to the AWS CLI from their own laptop in Russia. Why does the corporate firewall see no traffic? What log and specific keyword would you look for to detect this?

2.2 What API calls would attacker typical run after gaining initial access? What commands can the attacker run to find the crown jewels? What log and specific keyword would you look for to detect this?

2.3 The attacker, with admin keys, deletes the logs, S3 buckets and the snapshots. What API would the attacker call to perform these actions. What AWS control would have been able to prevent such activities?

- Email your answer to zongfu.chua@ntu.edu.sg with the Title "Week 3 Q2"



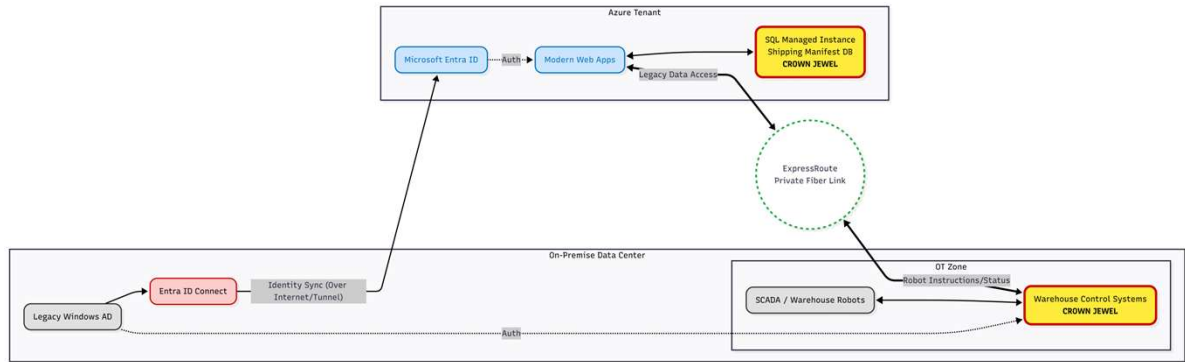
Exercise 3 : Hybrid Cloud ("GlobalLogistics")

GlobalLogistics is a shipping giant transitioning to the cloud. They operate a messy, complex hybrid environment.

- **On-Premise:** A massive data center running legacy Windows Active Directory (AD) and SCADA systems for warehouse robots.
- **Cloud:** An Azure tenant running modern Web Apps and SQL Managed Instances.
- **The Bridge:** The two worlds are connected by an **ExpressRoute** (a dedicated, private fiber link) and **Entra ID Connect** which syncs on-prem identities to the cloud.
- **The "Crown Jewels":** The Shipping Manifest Database (Azure) and the Warehouse Control Systems (On-Prem).



GlobalLogistics Data Flow Diagram





The Identity Synchronization Attack

Attackers have deployed ransomware on the On-Premise network. They have compromised a Domain Administrator account. Answer the following questions:

3.1 Can the ransomware move laterally through the ExpressRoute? Is traffic inside a private fiber link typically inspected by firewalls? What controls could potentially detect this?

3.2 If the attacker changes the password of the AD Admin on-prem, does that allow them to access the Azure Cloud Console? Explain why. What controls could potentially detect this?

3.3 If the attacker steals the ADFS Token Signing Certificate from the on-prem server, how can they bypass MFA to access Cloud applications? What controls could potentially detect this?

3.4 Could an isolated Identity account (Identity Bastion) prevent this on-prem compromise from destroying the cloud tenant?

3.5 You are a CISO trying to convince the board that we need to re-architect and adopt a zero trust network design. Perform PASTA analysis to pitch your point.

- Email your answer to zongfu.chua@ntu.edu.sg with the Title "Week 3 Q3"